CA Final Project Report

應用於多尺寸影像之 Sobel 邊緣偵測與二值化處理：GPU 加速與效能優化實作

313505011_葉家妤　　313510111_黃芷茵

Contents

I. Abstract ---------- 3

II. Introduction ---------- 4

A. 主題 ---------- 4
B. 實驗環境 ---------- 4
C. 檔案說明 ---------- 4
D. 組員分工 ---------- 6

III. 演算法內容 & Pipeline 切割方式 ---------- 7

A. 亂數生成結果 ---------- 3
- ✧ Edge_Detection_Binarization
- ✧ Edge_Detection_Binarization_Better
- ✧ Pipeline 切割方式

B. 真實圖片結果 ---------- 11
- ✧ Edge_Detection_Binarization_Image
- ✧ Pipeline 切割方式

IV. Simulation Result ---------- 14

A. 亂數生成結果 ---------- 14
- ✧ Edge_Detection_Binarization
- ✧ Edge_Detection_Binarization_Better

B. 真實圖片結果 ---------- 18
- ✧ Edge_Detection_Binarization_Image_AboutUs_Medium
- ✧ Edge_Detection_Binarization_Image_Cat_Large
- ✧ Edge_Detection_Binarization_Image_Cat_Small

V. Discussion ---------- 24

VI. Conclusions ---------- 27

VII. Reference ---------- 27

I. Abstract

Sobel 邊緣偵測與 Binarization 是影像前處理中常見且關鍵的技術，廣泛應用於商業領域如自駕車視覺、工業瑕疵檢測、醫療影像分析與文件掃描辨識等場景。在這些應用中，準確且快速的擷取邊緣資訊與圖像重點區域，對後續的辨識與分類演算法非常重要。因此本專題針對此類運算進行效能優化，希望能提升影像處理速度的同時，確保資訊準確性，以符合現代高效能、高即時性應用的需求。

在本次期末專題中，我們附上了五份完整的程式碼，每份皆包含基本的 Sobel 邊緣偵測版本，以及結合二值化（Binarization）處理的進階版本。主要目的是詳細呈現我們的優化流程，並透過完整的效能測試，驗證最終成果在各種情境下都能有效提升影像處理的速度與穩定性。

影像處理最核心的原則是「確保資訊正確，避免錯誤與遺失」，因此為了在優化演算法的同時驗證功能正確性，我們在處理真實影像前，先透過亂數生成矩陣（程式中定義為 SIZE*SIZE），進行邊緣偵測（Edge Detection）與二值化（Binarization）處理，並在此基礎上引入 pipeline 技術進行加速。透過觀察數值矩陣的變化，可有效確認 Sobel 與 Binarization 過程是否正確，進而確保即使進行效能優化，也能 100% 保持影像資料的完整性。

「亂數矩陣測試」的相關程式碼分別位於資料夾 Edge Detection Binarization 以及 Edge Detection Binarization Better 中。這兩者的主要差異在於：後者加入了針對不同矩陣大小（SIZE*SIZE）的條件判斷與對應的 pipeline 切割策略，根據輸入矩陣的大小自動選擇最適切的運算方式（使用 if else 達成），進一步提升整體處理效率。實驗結果也顯示「Edge Detection Binarization」雖然能正確運作，但在處理小尺寸（例如 SIZE 8x8）矩陣時無法有效發揮 pipeline 加速的優勢；而在「Edge Detection Binarization Better」中，即便處理從 8x8 到至少 4096x4096 的矩陣，都能在保持正確性的前提下，大幅提升執行速度。例如在 4096x4096 的矩陣測試中，未優化的 GPU1 執行時間為 178525.63 ms，優化後的 GPU2 則僅需 5.26 ms，優化效果非常顯著，且對於其他 SIZE 的矩陣測試中，執行速度業相較於「Edge Detection Binarization」佳。

在亂數矩陣測試完成後，我們進一步將優化技術應用至實際圖片。流程包括將圖片轉換為灰階圖像，分別套用 3x3、5x5、與 7x7 的 Sobel 運算子，再進行二值化處理。由於本次期末專題要求選擇「運算量大且可平行化的演算法」作為主題，因此我們選擇使用三張不同尺寸的真實圖片進行測試，以模擬實際應用情境。這三張圖片分別為：

- 組員合照（960x1706 pixels）
- 睡覺黑白貓貓（4032x3024 pixels）
- 嫌棄臉橘貓貓（500x500 pixels）

我們使用完全相同的程式碼，分別處理上述三張不同尺寸的圖片，目的在驗證我們設計的程式在面對不同解析度、長寬比與像素量級的影像時，皆能維持正確運作並有效提升運算效率。透過這樣的測試，可以明確證明我們的優化成果不僅具備高度的通用性與穩定性，同時也展現出良好的擴展能力。不僅符合本專題「運算量大且可平行化演算法」的核心要求，我們的設計也同時考量了對於小尺寸影像處理的優化，兼顧效率與準確性。

II. Introduction

A. 主題：應用於多尺寸影像之 Sobel 邊緣偵測與二值化處理：GPU 加速與效能優化實作

B. 實驗環境

使用課程提供的 Oracle VM VirtualBox 虛擬環境製作本次專題，規格如下：

- ❖ 作業系統版本：Ubnutu (64-bit)
- ❖ 主機板基本記憶體：1024 MB
- ❖ 處理器：1 CPU

C. 檔案說明

總共包含 5 包檔案，前 2 包是以亂數生成矩陣作為測資，進行 Sobel Edge Detection 以及 Binarization 處理；後 3 包是以真實圖片作為測資，進行 Sobel Edge Detection 以及 Binarization 處理。以下是每個檔案的簡述：

```
313505011_313510111
├ ReadMe.txt ----------------------------------> 資料夾內容概述
Edge_Detection_Binarization
        ├ ReadMe.txt ------------------------> 資料夾內容概述
        Binarization ┌ configure.ocelot -----> 用於 GPU 程式碼分析或模擬的設定檔
                     │                         ( Ocelot 是一個 CUDA 程式分析/模擬工具 )
                     ├ cuda1.cu ------------> 在GPU上實現Sobel & 二值化並採用第一種的切割方法
                     ├ cuda2.cu ------------> 在GPU上實現Sobel & 二值化並採用更優化的切割方法
                     ├ main.cu -------------> 程式主入口，實作 Sobel 邊緣偵測，分別在 CPU 與 GPU 上執行。
                     │                        其中GPU包含兩種方式(cuda1.cu 與 cuda2.cu)，透過 main.cu 呼叫執行
                     ├ Makefile ------------> 編譯所有 .cu 檔案。定義如 nvcc 的編譯方式、編譯參數、輸出執行檔名稱
                     ├ parameters.h --------> 定義block size常數(隨機生成的matrix尺寸)
                     └ ReadMe.txt ----------> 程式執行方式
        Normal       ┌ configure.ocelot -----> 同上
                     ├ cuda1.cu
                     ├ cuda2.cu
                     ├ main.cu
                     ├ Makefile
                     ├ parameters.h
                     └ ReadMe.txt
Edge_Detection_Binarization_Better
        ├ ReadMe.txt
        Binarization ┌ configure.ocelot -----> 同上
                     ├ cuda1.cu
                     ├ cuda2.cu
                     ├ main.cu
                     ├ Makefile
                     ├ parameters.h
                     └ ReadMe.txt
        Normal       ┌ configure.ocelot -----> 同上
                     ├ cuda1.cu
                     ├ cuda2.cu
                     ├ main.cu
                     ├ Makefile
                     ├ parameters.h
                     └ ReadMe.txt
```

```
Edge_Detection_Binarization_Image_AboutUs_Medium
    ReadMe.txt ------------------------> 資料夾內容概述
    Binarization
        configure.ocelot -----> 用於 GPU 程式碼分析或模擬的設定檔
                                (Ocelot 是一個 CUDA 程式分析/模擬工具)
        cuda1.cu -------------> 在GPU上實現Sobel & 二值化並採用第一種的切割方法
        cuda2.cu -------------> 在GPU上實現Sobel & 二值化並採用更優化的切割方法
        main.cu --------------> 程式主入口，實作 Sobel 邊緣偵測，分別在 CPU 與 GPU 上執行。
                                其中GPU包含兩種方式(cuda1.cu 與 cuda2.cu)，透過 main.cu 呼叫執行
        Makefile -------------> 編譯所有 .cu 檔案。定義如 nvcc 的編譯方式、編譯參數、輸出執行檔名稱
        ReadMe.txt ----------> 程式執行方式
        stb_image ------------> 圖片讀取的第三方函式庫，用於簡單讀寫 JPG 圖像
        test.jpg ---------------> 測試用圖片，會在 main.cu 中被讀入並套用 Sobel 邊緣偵測後輸出結果
    Normal
        configure.ocelot -----> 同上
        cuda1.cu
        cuda2.cu
        main.cu
        Makefile
        ReadMe.txt
        stb_image
        test.jpg
Edge_Detection_Binarization_Image_Cat_Large
    ReadMe.txt
    Binarization
        configure.ocelot -----> 同上
        cuda1.cu
        cuda2.cu
        main.cu
        Makefile
        ReadMe.txt
        stb_image
        test.jpg
    Normal
        configure.ocelot -----> 同上
        cuda1.cu
        cuda2.cu
        main.cu
        Makefile
        ReadMe.txt
        stb_image
        test.jpg
Edge_Detection_Binarization_Image_Cat_Small
    ReadMe.txt
    Binarization
        configure.ocelot -----> 同上
        cuda1.cu
        cuda2.cu
        main.cu
        Makefile
        ReadMe.txt
        stb_image
        test.jpg
    Normal
        configure.ocelot -----> 同上
        cuda1.cu
        cuda2.cu
        main.cu
        Makefile
        ReadMe.txt
        stb_image
        test.jpg
```

D. 組員分工

本期末專題由兩位組員共同合作完成，以下為分工內容：

工作內容	負責人
CPU 設計	葉家妤
GPU 1 / 2 設計 & 優化	黃芷茵 & 葉家妤（共同完成）
結報	黃芷茵

III. 演算法內容 & Pipeline 切割方式

（此處簡單的解釋程式碼意義，並且進行程式碼比較，呈現為何能使影像處理的速度更優化）

A. 亂數生成結果

✧ Edge_Detection_Binarization

本專題實作兩版 CUDA GPU 平行化 Sobel 邊緣偵測與二值化處理核心，分別為基本版（CUDA1）與優化版（CUDA2），兩者皆採用 pipeline 概念進行資料分配。以下說明差異：

❖ Sobel 邊緣偵測 kernel 比較：

CUDA1：條件式 Pipeline 資料分配

```
__global__ void sobelCUDA1(int* image, int* result){

    int totalThreads = blockDim.x * gridDim.x;
    int totalPixels = SIZE * SIZE;
    int stripe = totalPixels/totalThreads;

    int head = (blockIdx.x * blockDim.x + threadIdx.x) * stripe;
    int limit = head + stripe;

    int sobelX[3][3] = {
        {-1, 0, 1},
        {-2, 0, 2},
        {-1, 0, 1}
    };

    int sobelY[3][3] = {
        {1, 2, 1},
        {0, 0, 0},
        {-1, -2, -1}
    };

    int gx = 0;
    int gy = 0;

    for(int i =head;i<limit;i++){
        int row = i / SIZE;
        int col = i % SIZE;
        for(int m=0;m<3;m++){
            for(int n=0;n<3;n++){
                int r = row + m;
                int c = col + n;
                int pixel = image[(r)*(SIZE+2)+(c)];
                gx += pixel * sobelX[m][n];
                gy += pixel * sobelY[m][n];
            }
        }
        int mag = sqrtf((float)(gx * gx + gy * gy));
        if(mag >128)mag = 255;
        if(mag <128)mag = 0;
        result[row*SIZE + col]=mag;
    }
}
```

將所有像素分為等寬的 stripe，並以 head 和 limit 控制每個 thread 所處理的像素區間。接著每個 thread 根據 thread ID 計算其負責的像素區間，並於該區間內手動套用 3x3 Sobel mask。最後利用 gx 與 gy 累加計算水平、垂直方向梯度，再透過平方根計算邊緣強度並進行閾值化（Binary Thresholding）。

CUDA2：優化版，整合 Pipeline 與迴圈移動

```
__global__ void sobelCUDA2(int* image, int* image_sobel){

    int totalThreads = blockDim.x * gridDim.x;
    int totalPixels = SIZE * SIZE;
    int stripe = totalPixels/totalThreads;
    int head = (blockIdx.x * blockDim.x + threadIdx.x) * stripe;
    int limit = head + stripe;

    int sobelX[9] = {
        -1, 0, 1,
        -2, 0, 2,
        -1, 0, 1
    };

    int sobelY[9] = {
         1,  2, 1,
         0,  0,  0,
        -1, -2, -1
    };

    int gx = 0;
    int gy = 0;

    for(int i=head;i<SIZE*SIZE;i+=stripe){
        int row = i / SIZE;
        int col = i % SIZE;

        for(int m=0;m<3;m++){
            for(int n=0;n<3;n++){
                int pixel = image[(row + m)*(SIZE+2)+(col+n)];
                gx += pixel * sobelX[m*3 + n];
                gy += pixel * sobelY[m*3 + n];
            }
        }
        int mag = sqrtf((float)(gx*gx + gy*gy));
        if(mag >128)mag = 255;
        if(mag<128)mag = 0;
        image_sobel[i]=mag;
    }
}
```

相較於 CUDA1，此版本改以「for 迴圈遞增 i += stripe」的方式掃描像素，可動態跳躍處理分配區間，更具延展性與效能彈性。此處將 Sobel kernel 展平成一維陣列 sobelX[9] / sobelY[9]，如此可以減少 2D array indexing overhead。最後使用和 CUDA1 相同的邊緣強度與二值化方法，確保偵測準確性。

→ Pipeline 設計：兩者皆使用「stripe 分段處理 ＋for 迴圈」分配 thread 工作量，避免 thread 空轉。此外，CUDA2 以 i += stripe 設計，有更好的可擴展性，能根據輸入資料量彈性擴充。

→ Indexing 優化：CUDA2 透過一維展開的 Sobel mask 陣列，能降低記憶體讀取與計算指令數，進而優化效能。

→ 皆使用 dim3 dimGrid(4);dim3 dimBlock(8);，表示每個 Block 中有 8 個 Threads，也就是每個 Block 裡有 8 條平行執行的 thread），又整個 GPU 上會開啟 4 個 Block，也就是這個 kernel 會在 GPU 上開啟 4 個 block、每個 block 裡有 8 個 thread → 總共 32 個 thread。

✧ Edge_Detection_Binarization_Better

因為此程式是我們藉由上面的 Edge_Detection_Binarization 再優化而來的，因此我將此 CUDA2 程式碼與剛剛 Edge_Detection_Binarization 的 CUDA2 進行比較，主要不同之處在於主程式「GPU_kernel_2()」不同，Edge_Detection_Binarization_Better CUDA2 會依據輸入的圖像大小不同來進行動態調整 block/grid。

❖ 依照圖像大小動態調整 block/grid 設定：

Edge_Detection_Binarization CUDA2

```
float GPU_kernel_2(int* image, int* result_sobel){
    int* d_img;
    int* d_result;

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaMalloc((void**)&d_img, sizeof(int) * (SIZE+2) * (SIZE+2));
    cudaMalloc((void**)&d_result, sizeof(int) * (SIZE) * (SIZE));

    cudaMemcpy(result_sobel, d_result, sizeof(int) * SIZE *SIZE, cudaMemcpyDeviceToHost
    cudaMemset(d_result, 0, sizeof(int)*SIZE*SIZE);

    cudaEventRecord(start,0);
    dim3 dimGrid(4);
    dim3 dimBlock(8);

    sobelCUDA2<<<dimGrid, dimBlock>>>(d_img, d_result);
    cudaEventRecord(stop, 0);
    cudaEventSynchronize(stop);

    cudaMemcpy(result_sobel, d_result, sizeof(int) * SIZE *SIZE, cudaMemcpyDeviceToHost
    cudaFree(d_img);
    cudaFree(d_result);

    float elapsedTime;
    cudaEventElapsedTime(&elapsedTime, start, stop);
    return elapsedTime;
}
```

Edge_Detection_Binarization_Better CUDA2

```
float GPU_kernel_2(int* image, int* result_sobel){
if(SIZE>=64){
    int* d_img;
    int* d_result;

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaMalloc((void**)&d_img, sizeof(int) * (SIZE+2) * (SIZE+2));
    cudaMalloc((void**)&d_result, sizeof(int) * (SIZE) * (SIZE));

    cudaMemcpy(result_sobel, d_result, sizeof(int) * SIZE *SIZE, cudaMemcpyDeviceToHost
    cudaMemset(d_result, 0, sizeof(int)*SIZE*SIZE);

    cudaEventRecord(start,0);

    dim3 dimGrid(4);
    dim3 dimBlock(4);

    sobelCUDA2<<<dimGrid, dimBlock>>>(d_img, d_result);
    cudaEventRecord(stop, 0);
    cudaEventSynchronize(stop);

    cudaMemcpy(result_sobel, d_result, sizeof(int) * SIZE *SIZE, cudaMemcpyDeviceToHost
    cudaFree(d_img);
    cudaFree(d_result);

    float elapsedTime;
    cudaEventElapsedTime(&elapsedTime, start, stop);
    return elapsedTime;
}
else{
int* d_img;
    int* d_result;

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaMalloc((void**)&d_img, sizeof(int) * (SIZE+2) * (SIZE+2));
    cudaMalloc((void**)&d_result, sizeof(int) * (SIZE) * (SIZE));

   cudaMemcpy(result_sobel, d_result, sizeof(int) * SIZE *SIZE, cudaMemcpyDeviceToHost
   cudaMemset(d_result, 0, sizeof(int)*SIZE*SIZE);

    cudaEventRecord(start,0);

    dim3 dimGrid(2);
    dim3 dimBlock(2);

    sobelCUDA2<<<dimGrid, dimBlock>>>(d_img, d_result);
    cudaEventRecord(stop, 0);
    cudaEventSynchronize(stop);

    cudaMemcpy(result_sobel, d_result, sizeof(int) * SIZE *SIZE, cudaMemcpyDeviceToHost
    cudaFree(d_img);
    cudaFree(d_result);

    float elapsedTime;
    cudaEventElapsedTime(&elapsedTime, start, stop);
    return elapsedTime;
    }
}
```

❖ Edge_Detection_Binarization_Better & Edge_Detection_Binarization CUDA2 比較：

	Edge_Detection_Binarization	**Edge_Detection_Binarization_Better**
kernel 本體	兩者 kernel 本體完全相同	
block/grid	固定為 dimGrid(4), dimBlock(8)	依照圖像大小動態調整 If (SIZE >= 64) dimGrid(4), dimBlock(4) else dimGrid(2), dimBlock(2)
記憶體&計時	固定流程	依圖大小有不同邏輯，使小圖 overhead↓
適合圖像尺寸	較大尺寸較佳	都行，針對小圖、大圖各自最佳化

→ Better 為不同影像尺寸選擇更合適的 thread 數量與佈局，避免 thread 數太多造成 idle（尤其是小圖）、或太少導致 under-utilization（大圖）。

→ Better 更貼近 memory throughput 與 occupancy 平衡：小圖使用少量 thread，減少 kernel launch 的 scheduling 花費；大圖用較多 thread，但避免過多導致 thread starvation。

✧ Pipeline 切割方式

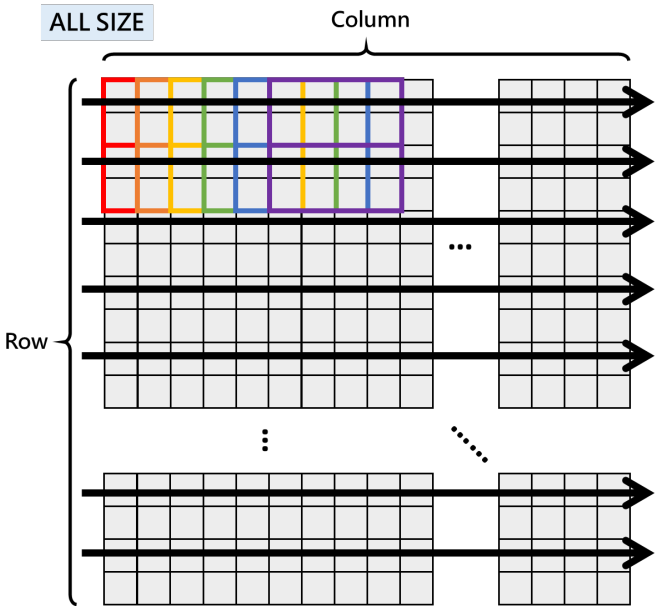
此處將「Edge Detection Binarization」的 CUDA 1 定義為 CUDA 1；「Edge Detection Binarization」的 CUDA 2 定義為 CUDA 2；「Edge Detection Binarization Better」的 CUDA 2 定義為 CUDA 2_1。將各自介紹每個的 Pipeline 切割方式，並將三個逐漸優化的過程製作成表格，方便解釋和理解：

❖ Pipeline 切割方式比較表格

	CUDA 1	CUDA 2	CUDA 2_1
處理單位	橫向 row 處理	stripe 切段，平均分配	條件式 stripe 分段
pipeline	一條 row 一條 row 做	每個 thread 一條連續 stripe	對應大小 選擇最佳 stripe 數
memory access	row-major 易衝突	分散記憶體 bank	分散記憶體 bank 且動態調整
bank conflict	易發生	低	低、adaptive
平行度	較差	穩定	更穩定且廣泛
適合圖片大小	小圖	大圖	全尺寸適用

❖ 「Edge Detection Binarization」的 CUDA 1 → CUDA 1（以 row 為單位處理的方式）

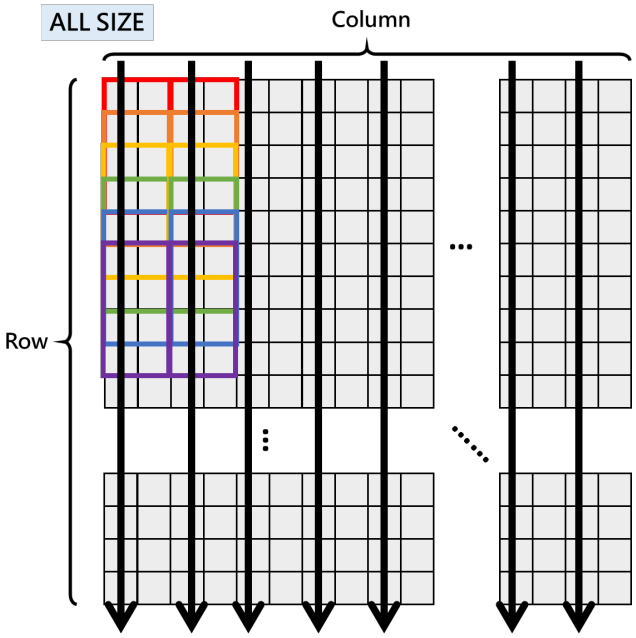
CUDA1 每個 thread 根據自己的 (blockIdx.x * blockDim.x + threadIdx.x) 決定起始 pixel ID，然後**以一個「row 為單位」依序處理，完成這個 row 之後，再進入下一個 row**。平行度來自「thread 在同一 row 中分工處理不同 column」。資料處理示意圖如下：



❖ 「Edge Detection Binarization」的 CUDA 2 → CUDA 2（以一段 stripe 為單位處理）

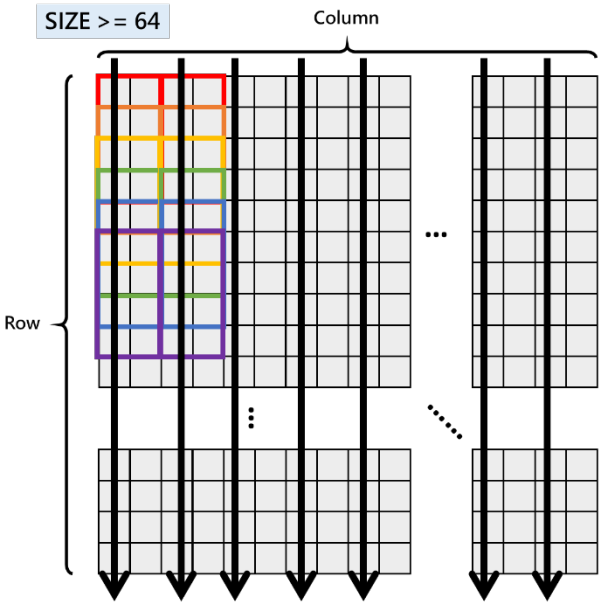
CUDA2 以一段 stripe 為單位處理。每個 thread 根據 thread ID（blockIdx、threadIdx）計算要處理的 pixel 起始點 head，並處理 stripe 個像素。將整張影像平均切成 32 段 stripe

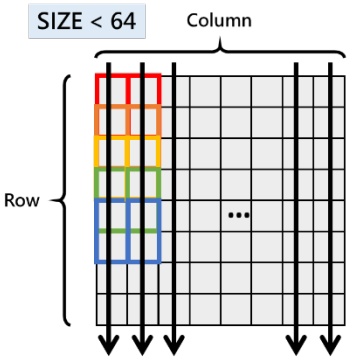
（dimGrid(4), dimBlock(8)），每個 thread 處理 32 個 pixel。**處理順序會從不同 row、不同 col 中取樣，呈現「Z 字型」間隔**。每個 thread 處理一整段連續的工作，因此有良好的 instruction locality。相較於 CUDA1 更適合中型或大型圖片，平行化分段穩定。資料處理示意圖如下：



❖ 「Edge Detection Binarization Better」CUDA 2 → CUDA 2_1（動態調整分段與佈局）

延續 CUDA2 的 thread striping 方式，但加入了動態條件判斷，根據圖片大小調整 grid/block 數量，Kernel 邏輯仍使用和 CUDA2 相同的 head + stripe 分配方式。圖片大的情況使用更多 thread（16 個），圖片小的情況使用較少 thread（4 個），避免啟動 overhead 和浪費。動態佈局讓分工數量更貼近實際工作量，提升整體 resource utilization。






```
if (SIZE >= 64)
    dim3 dimGrid(4); dim3 dimBlock(4);
else
    dim3 dimGrid(2); dim3 dimBlock(2);
```

B. 真實圖片結果

以下為關於真實圖片模擬相關說明：

一般而言，執行 Sobel 邊緣偵測時常會使用 OpenCV 套件。然而，在本次練習中，我們選擇改用更輕量的 stb_image.h，手動實作 Sobel 邊緣偵測演算法。透過自行撰寫 Sobel 運算子，並比較 3x3、5x5、與 7x7 三種不同矩陣的運作方式與結果，使我們對邊緣偵測的基本處理流程與背後的數學原理有更深入的理解。

此外，選擇 stb 套件也有實務上的優勢。在虛擬機環境中安裝 OpenCV 可能會遇到版本不相容或安裝繁瑣等問題，而 stb 為單一 .h 檔案，不僅整合容易、無需額外編譯，也支援多種常見彩色圖像格式（如 PNG、JPEG、BMP），更方便實作與測試。

在此專題中我們的影像處理流程如下：

- ❖ 原始圖像為彩色 .jpg 檔案，為三通道影像，資料結構為 [height][width][3]，展開為 1D 向量長度為 width × height × 3
- ❖ 經灰階轉換後，成為單通道影像，轉為長度為 width × height 的 1D 向量
- ❖ 套用 Sobel 邊緣偵測後，仍為 [height][width] 的灰階圖，但每個像素值代表邊緣強度
- ❖ 最終以.pgm 格式儲存，實際為輸出一個長度為 width × height 的灰階像素陣列（byte 值）

✧ Edge_Detection_Binarization_Image

這裡的 CUDA1、CUDA2 程式碼是由剛剛介紹過的 Edge_Detection_Binarization_Better 的 CUDA1、CUDA2 加入讀取真實相片，以及針對相片進行切割處理而來，只是因為讀取數值和真實圖片需要經過的處理不同，因此有些微的調整。比較需要注意的調整是在 CUDA2 的部分還額外加入了 shared memory 的概念，加速讀取與重複使用 pixel。

以下將介紹讀取真實圖片的 CUDA1、CUDA2 程式碼比較：

❖ Edge_Detection_Binarization_Image CUDA1、CUDA2 比較表格及相對應的程式碼

	CUDA1	CUDA2
處理單位	stripe 切段，thread 處理多像素	每 thread 處理一 pixel

	`int stripe = (totalPixels + totalThreads - 1) / totalThreads;`
	`int head = globalThreadId * stripe;`
	`int limit = min(head + stripe, totalPixels);`
	`for (int idx = head; idx < limit; ++idx) {`
	`  ...`
	`}`
		`int x = blockIdx.x * blockDim.x + threadIdx.x;`
		`int y = blockIdx.y * blockDim.y + threadIdx.y;`

Pipeline	stripe 型、無 tile	tile-based 區塊 pipeline

	`for (int idx = head; idx < limit; ++idx) {`
	`  ...`
	`}`
		`int row_o = blockIdx.y * TILE_WIDTH + ty;`
		`int col_o = blockIdx.x * TILE_WIDTH + tx;`

Memory access	全用 global memory	shared memory + constant

	`int px = input[(y - offset + i) * width + (x - offset + j)];`	`__shared__ unsigned char smem[TILE_WIDTH + MAX_KERNEL_SIZE - 1][TILE_WIDTH + MAX_KERNEL_SIZE - 1];`
		`...`
		`smem[ty][tx] = input[row_i * width + col_i];`

Kernel 權重	global memory 傳入	constant memory 儲存
	`cudaMemcpy(d_kernelX, kernelX_host, kernel_len * sizeof(int), cudaMemcpyHostToDevice);`	`__constant__ int SOBEL_X_7x7[MAX_KERNEL_SIZE * MAX_KERNEL_SIZE];` `cudaMemcpyToSymbol(SOBEL_X_7x7, kernelX_host, kernel_len * sizeof(int));`

在此處 CUDA2 相較於 CUDA1 較好是因為 CUDA2 有更佳記憶體存取模式、有效利用 shared memory、constant memory 較快而且共用、可以根據需求選擇更快邏輯。

✧ Pipeline 切割方式

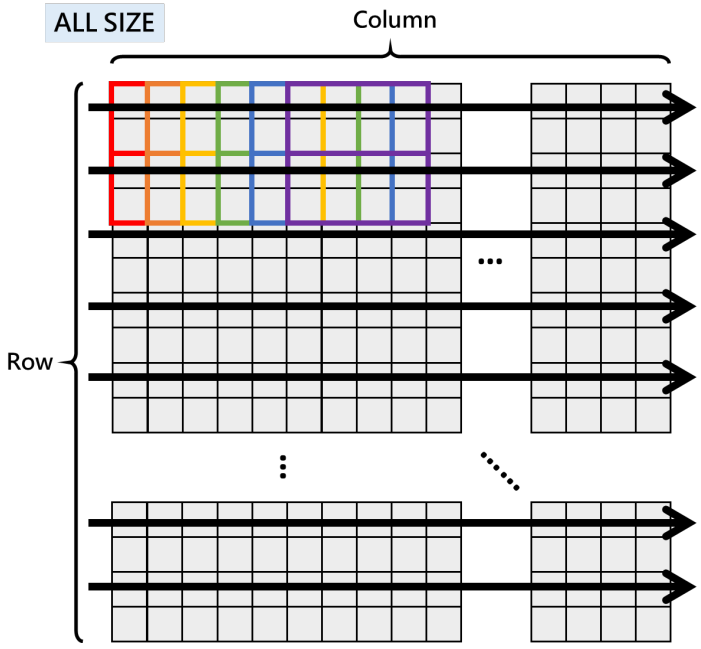
此處將「Edge Detection Binarization Image」的 CUDA 1 定義為 CUDA 1；「Edge Detection Binarization Image」的 CUDA 2 定義為 CUDA 2。將各自介紹每個的 Pipeline 切割方式，並將逐漸優化的過程製作成表格，方便解釋和理解：

❖ Pipeline 切割方式比較表格

	CUDA 1_stripe	**CUDA 2_tile**
切割方式	依照像素編號切成 N 段	以空間為單位劃分為 tile 區塊
分工方式	每 thread 處理一段畫素	每 block 處理一塊 tile，thread 處理 tile 內 pixel
想像圖像	一條長條分段（線性）	2D 方格 map 分塊（田字格）
memory access	全部 global memory	使用 shared memory
Pipeline 視覺感	像工廠輸送帶切段	像一群人在整理各自的區域

❖ 「Edge Detection Binarization Image」的 CUDA 1 → CUDA 1（linear stripe 分段法）

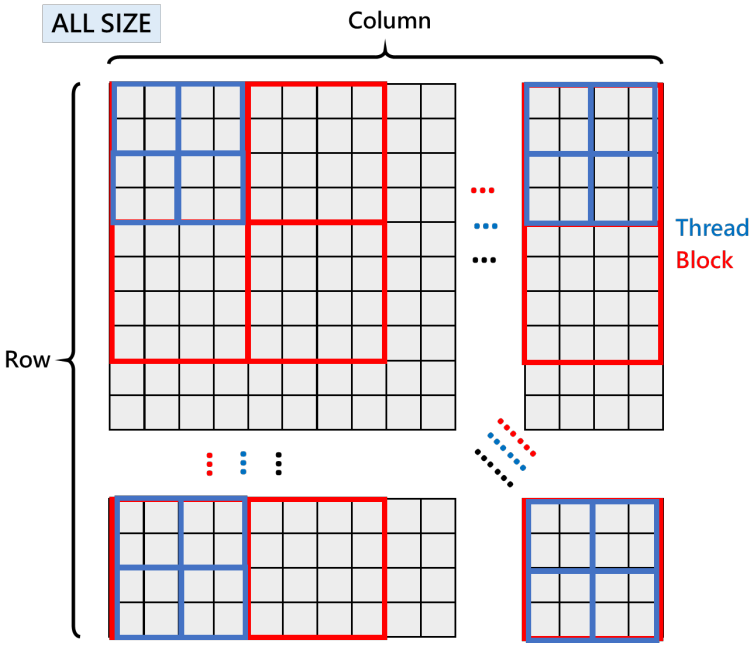
CUDA1 把整張圖片當作一條很長的像素陣列（flatten 成 1D）。將這個像素序列切成 N 段 stripe，每一段由一個 thread 處理。每個 thread 用 globalThreadId 確定自己是哪一段的負責人（head ~ limit），然後逐一 for-loop 計算這段的每個像素，但是這種 pipeline 沒有空間區域性，不知道哪些 pixel 鄰近，導致記憶體不連續存取。資料處理示意圖如下：



❖ 「Edge Detection Binarization Image」的 CUDA 2 → CUDA 2（tile-based block pipeline）

CUDA2 把圖片視為一張 2D 地圖，每個 block 就像是一塊小田地。每個 block 處理一塊固定區域就是 tile。每個 block 的 thread 會先把這塊 tile + 邊界擴張（padding）載入 shared memory，然後 pipeline 內部用 shared memory 做內部運算。

這種方式相對於 CUDA1 的優點有 tile 有空間區域性（鄰近的 pixel 一起處理）、shared memory 加速讀取與重複使用 pixel。資料處理示意圖如下：



IV. Simulation Result

在進行小尺寸圖像處理時，我們觀察到少數情況下優化後的 GPU2 執行時間反而略慢於 GPU1（約 10 次中出現 1 次）。我們認為這是合理現象。

原因可能包括：CUDA kernel 並非每次執行都能獲得相同的 GPU 資源，可能受到背景程式使用 GPU 或前一次 kernel 執行尚未完全釋放資源的影響。此外當輸入資料過小（例如 SIZE = 8，4 block * 8 thread = 32 thread 下），可能無法充分利用 GPU 計算資源，導致 pipeline 技術無法發揮明顯優勢，使效能差異不易觀察。

因此，我們在處理小尺寸資料時，採取多次重複測量並取平均的方式，以獲得更具代表性且具參考價值的比較結果。

A. 亂數生成結果

✧ Edge_Detection_Binarization

❖ 亂數生成的矩陣經過 CPU、GPU 1、GPU 2 影像處理後的結果 (此處以 8x8 矩陣為例，較方便觀察結果是否正確)：

Normal（Sobel 邊緣偵測）

```
===========image=============
+0    +0    +0    +0    +0    +0    +0    +0    +0    +0
+0    +103  +198  +105  +115  +81   +255  +74   +236  +0
+0    +41   +205  +186  +171  +242  +251  +227  +70   +0
+0    +124  +194  +84   +248  +27   +232  +231  +141  +0
+0    +118  +90   +46   +99   +51   +159  +201  +154  +0
+0    +102  +50   +13   +183  +49   +88   +163  +90   +0
+0    +37   +93   +5    +23   +88   +233  +94   +212  +0
+0    +171  +178  +205  +198  +155  +180  +84   +17   +0
+0    +14   +130  +116  +65   +33   +61   +220  +135  +0
+0    +0    +0    +0    +0    +0    +0    +0    +0    +0

========CPU========

========after sobel=======
+255  +255  +255  +255  +255  +255  +255  +255
+255  +252  +130  +193  +255  +176  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +179  +165  +255  +255  +255
+255  +255  +255  +223  +107  +255  +84   +255
+255  +255  +255  +255  +255  +218  +246  +255
+255  +212  +255  +172  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255

========GPU========

========GPU1=======
+255  +255  +255  +255  +255  +255  +255  +255
+255  +252  +130  +193  +255  +176  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +179  +165  +255  +255  +255
+255  +255  +255  +223  +107  +255  +84   +255
+255  +255  +255  +255  +255  +218  +246  +255
+255  +212  +255  +172  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255

========GPU2=======
+255  +255  +255  +255  +255  +255  +255  +255
+255  +252  +130  +193  +255  +176  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +179  +165  +255  +255  +255
+255  +255  +255  +223  +107  +255  +84   +255
+255  +255  +255  +255  +255  +218  +246  +255
+255  +212  +255  +172  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255

==================================
GPU1 =  4.02 ms
GPU2 = 11.44 ms
CPU = 0.051900 ms
```

Binarization（邊緣偵測+Binarization）

```
===========image=============
+0    +0    +0    +0    +0    +0    +0    +0    +0    +0
+0    +103  +198  +105  +115  +81   +255  +74   +236  +0
+0    +41   +205  +186  +171  +242  +251  +227  +70   +0
+0    +124  +194  +84   +248  +27   +232  +231  +141  +0
+0    +118  +90   +46   +99   +51   +159  +201  +154  +0
+0    +102  +50   +13   +183  +49   +88   +163  +90   +0
+0    +37   +93   +5    +23   +88   +233  +94   +212  +0
+0    +171  +178  +205  +198  +155  +180  +84   +17   +0
+0    +14   +130  +116  +65   +33   +61   +220  +135  +0
+0    +0    +0    +0    +0    +0    +0    +0    +0    +0

========CPU========

========after sobel=======
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +0   +255  +0   +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255

========GPU========

========GPU1=======
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +0   +255  +0   +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255

========GPU2=======
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +0   +255  +0   +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255
+255  +255  +255  +255  +255  +255  +255  +255

==================================
GPU1 =  4.07 ms
GPU2 = 11.40 ms
CPU = 0.050900 ms
```

左邊為資料夾「Normal」中結果，僅進行 Sobel 邊緣偵測；右邊為資料夾「Binarization」中結果，進行 Sobel 邊緣偵測後再進行二值化處理。

在「Edge_Detection_Binarization」中，當處理的矩陣尺寸較小（如此處的 8x8 矩陣），因為 cuda2 使用的是 stripe-based 分配（每個 thread 處理多個像素），這僅對於大圖像效果好，

因為可以減少 thread 數目與分支不均。但是在小圖像時，stripe 效果反而拖累效能，因為可能造成每個 thread 處理的 pixel 太少，不值得使用，且相較於 cuda1 又多了迴圈與條件判斷的 overhead，因此此處看起來 GPU2 相較於 GPU1 慢很多，這是符合我們設計預期的。這項問題會在後續的「Edge_Detection_Binarization_Better」被成功優化。

❖ 「Edge_Detection_Binarization」運行速度總表，進行多次測量後取平均為最終結果：

Edge_Detection_Binarization							
Normal				Binarization			
Size	GPU1 (ms)	GPU2 (ms)	CPU (ms)	Size	GPU1 (ms)	GPU2 (ms)	CPU (ms)
8*8	4.02	11.44	0.05	8*8	4.07	11.40	0.05
	3.82	11.31	0.06		4.03	14.03	0.06
	3.96	11.29	0.05		3.99	13.45	0.05
	3.85	11.24	0.07		4.23	13.46	0.05
	3.89	11.59	0.06		10.15	7.89	0.05
	3.92	11.76	0.06		9.91	7.80	0.05
	4.06	11.94	0.06		9.59	7.81	0.05
	3.88	11.31	0.05		3.97	13.49	0.06
AVG	**3.93**	**11.49**	**0.06**	**AVG**	**6.24**	**11.17**	**0.05**
32*32	20.28	7.81	0.71	32*32	13.85	8.01	0.56
	20.06	7.78	0.62		21.05	7.85	0.56
	20.74	7.79	0.62		14.05	8.44	0.56
AVG	**20.36**	**7.79**	**0.65**	**AVG**	**16.32**	**8.10**	**0.56**
64*64	51.65	7.88	2.43	64*64	56.41	7.90	2.21
	50.67	7.77	2.43		55.12	7.98	2.21
	51.10	7.83	2.42		55.32	7.75	2.28
AVG	**51.14**	**7.83**	**2.43**	**AVG**	**55.62**	**7.88**	**2.23**
512*512	2312.12	8.03	219.21	512*512	2440.31	8.08	326.44
	2277.12	7.85	210.91		2344.13	8.73	257.44
	2308.76	7.87	229.14		2410.99	8.05	401.81
AVG	**2299.33**	**7.92**	**219.76**	**AVG**	**2398.48**	**8.29**	**328.56**
4096*4096	143651.34	8.13	1813.35	4096*4096	142170.55	7.94	557.94
	144536.25	12.49	1656.21		144450.78	8.24	536.05
	144012.02	8.07	1609.36		145630.56	14.06	705.61
AVG	**144066.54**	**9.56**	**1692.97**	**AVG**	**144083.96**	**10.08**	**599.87**

左邊為資料夾「Normal」中結果，僅進行 Sobel 邊緣偵測；右邊為資料夾「Binarization」中結果，進行 Sobel 邊緣偵測後再進行二值化處理。

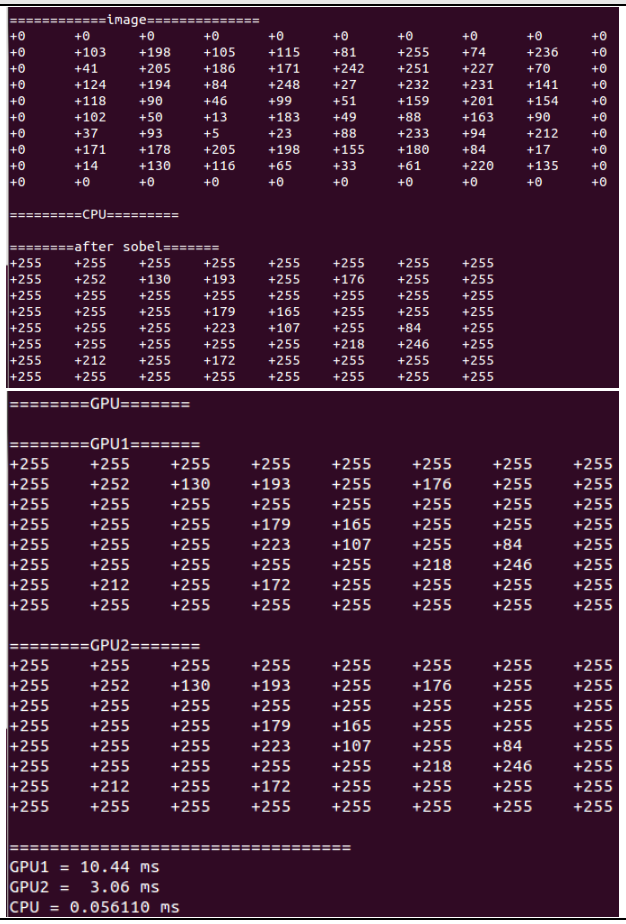
❖ 「Edge_Detection_Binarization」運行速度截圖（以 Normal 為例）：

8x8（SIZE < 64）	32x32（SIZE < 64）
GPU1 = 4.02 ms GPU2 = 11.44 ms CPU = 0.051900 ms	GPU1 = 20.28 ms GPU2 = 7.81 ms CPU = 0.708530 ms
64x64（SIZE >= 64）	**4096x4096（SIZE >= 64）**
GPU1 = 51.65 ms GPU2 = 7.88 ms CPU = 2.434760 ms	GPU1 = 143651.34 ms GPU2 = 8.13 ms CPU = 1813.350170 ms

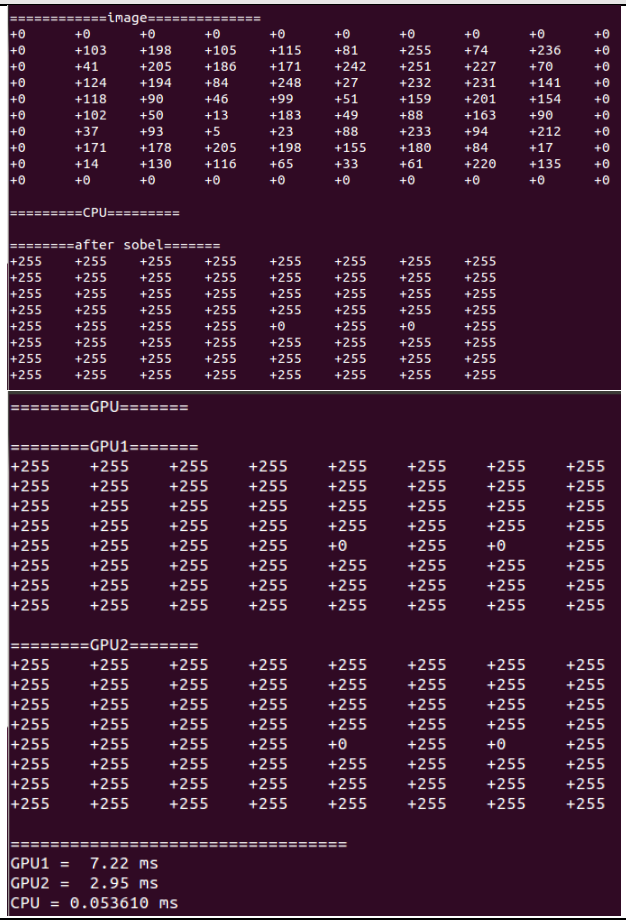
✧ Edge_Detection_Binarization_Better

❖ 亂數生成的矩陣經過 CPU、GPU 1、GPU 2 影像處理後的結果 (此處以 8x8 矩陣為例，較方便觀察結果是否正確)：

Normal（Sobel 邊緣偵測）



Binarization（邊緣偵測+Binarization）



左邊為資料夾「Normal」中結果，僅進行 Sobel 邊緣偵測；右邊為資料夾「Binarization」中結果，進行 Sobel 邊緣偵測後再進行二值化處理。

可以看到此處成功處理的矩陣尺寸較小（8x8 矩陣）的優化，且效果明顯。利用判斷不同的矩陣大小，有不同的資料運算方式，判斷內容如下：

圖像尺寸區間	dimGrid	dimBlock	說明
If (SIZE >= 64)	(4, 1)	(4, 1)	16 threads，適合大圖，多數 GPU 能處理
else	(2, 1)	(2, 1)	4 threads，更有效使用 GPU，同時不會在處理小圖時過度浪費資源

❖ 「Edge_Detection_Binarization_Better」運行速度總表：

Edge_Detection_Binarization_Better							
Normal				Binarization			
Size	GPU1 (ms)	GPU2 (ms)	CPU (ms)	Size	GPU1 (ms)	GPU2 (ms)	CPU (ms)
8*8	10.44	3.06	0.06	8*8	7.22	2.95	0.05
	9.84	2.99	0.06		4.13	8.29	0.06
	4.15	8.54	0.05		4.12	8.46	0.06
	4.32	8.25	0.05		9.73	2.97	0.05
	9.65	3.07	0.06		10.58	3.15	0.06
	4.07	8.36	0.05		9.93	3.08	0.05
	9.82	2.96	0.06		21.55	3.06	0.05
	4.13	8.73	0.06		4.00	8.44	0.06
AVG	**7.05**	**5.75**	**0.05**	**AVG**	**8.91**	**5.05**	**0.05**
32*32	26.52	2.92	0.62	32*32	24.52	3.34	0.57
	17.62	2.90	0.62		24.89	2.94	0.77
	21.28	3.03	0.62		24.59	3.01	0.57
AVG	**21.81**	**2.95**	**0.62**	**AVG**	**24.67**	**3.10**	**0.64**
64*64	66.15	4.35	2.49	64*64	60.67	4.49	2.21
	66.83	4.48	2.46		61.34	4.44	2.22
	66.04	4.38	2.42		60.36	4.35	2.45
AVG	**66.34**	**4.40**	**2.46**	**AVG**	**60.79**	**4.43**	**2.29**
512*512	2851.66	4.49	291.33	512*512	2857.58	4.50	392.06
	2847.42	4.58	362.70		2771.75	11.50	306.52
	2842.03	4.62	360.57		2840.77	4.50	290.84
AVG	**2847.04**	**4.56**	**338.20**	**AVG**	**2823.37**	**6.83**	**329.80**
4096*4096	178410.53	6.14	1510.54	4096*4096	176941.70	4.58	791.87
	178425.84	4.95	1482.87		176745.16	4.76	740.51
	178740.53	4.70	1646.52		177801.97	4.65	751.12
AVG	**178525.63**	**5.26**	**1546.64**	**AVG**	**177162.94**	**4.66**	**761.17**

左邊為資料夾「Normal」中結果，僅進行 Sobel 邊緣偵測；右邊為資料夾「Binarization」中結果，進行 Sobel 邊緣偵測後再進行二值化處理。

每個尺寸進行多次測量後取平均為最終結果。可以看到我們在此處成功利用 pipeline 的方式切割資料，且新增的「先由判斷 SIZE 大小再決定使用何種 pipeline 方式」非常成功，達成在所有尺寸下（8x8 矩陣一直到至少 4096x4096 矩陣）使 GPU2 加速的目的。

❖ 「Edge_Detection_Binarization_Better」運行速度截圖（以 Normal 為例）：

8x8（SIZE < 64）	32x32（SIZE < 64）
GPU1 = 10.44 ms GPU2 = 3.06 ms CPU = 0.056110 ms	GPU1 = 25.89 ms GPU2 = 2.94 ms CPU = 0.616660 ms
64x64（SIZE >= 64）	**4096x4096（SIZE >= 64）**
GPU1 = 66.15 ms GPU2 = 4.35 ms CPU = 2.485660 ms	GPU1 = 178410.53 ms GPU2 = 6.14 ms CPU = 1510.537250 ms

B. 真實圖片結果

✧ Edge_Detection_Binarization_Image_AboutUs_Medium

❖ 真實圖片（960*1706 pixels）經過 CPU、GPU 1、GPU 2 影像處理後的結果：

1. Normal（Sobel 邊緣偵測）

Sobel Operator	CPU	GPU 1	GPU2
真實圖片			
3x3			
5x5			
7x7			

2. Binarization（邊緣偵測+Binarization）

Sobel Operator	CPU	GPU 1	GPU2
真實圖片			
3x3			
5x5			
7x7			

由以上圖片可以看到，CPU、GPU1、GPU2 的模擬圖片都相同，成功進行影像處理加速的同時也確保資料不會丟失、誤傳。

❖ 「Edge_Detection_Binarization_Image_AboutUs_Medium」運行速度總表：

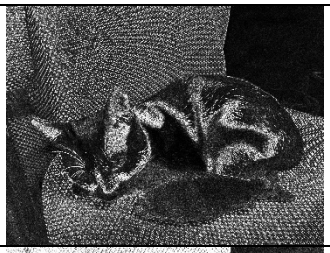
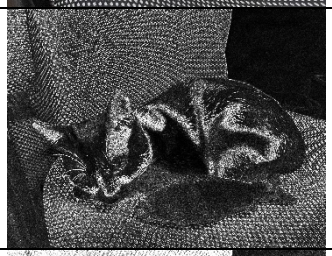
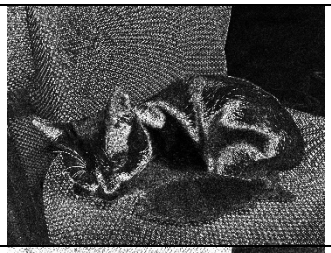
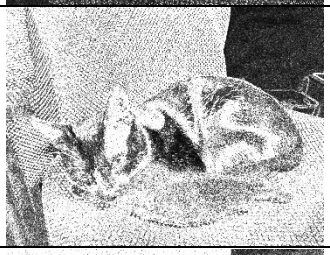
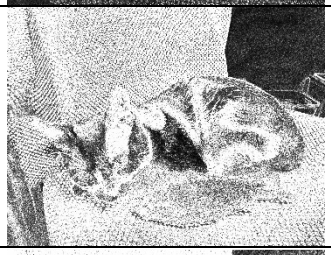
Edge_Detection_Binarization_Image_AboutUs_Medium									
Normal					Binarization				
Pixel	Sobel Operator	GPU1 (ms)	GPU2 (ms)	CPU (ms)	Pixel	Sobel Operator	GPU1 (ms)	GPU2 (ms)	CPU (ms)
960*1706	3×3	64686.56	20926.75	47.69	960*1706	3×3	61906.40	20354.83	11.00
	5×5	135059.92	89579.30	17.80		5×5	135643.11	89312.98	13.17
	7×7	237563.75	122703.62	17.76		7×7	235932.75	122778.44	16.85
AVG		145770.08	77736.56	27.75	AVG		144494.08	77482.08	13.68

左邊為資料夾「Normal」中結果，僅進行 Sobel 邊緣偵測；右邊為資料夾「Binarization」中結果，進行 Sobel 邊緣偵測後再進行二值化處理，可以看到 GPU2 的速度相較於 GPU1 都快了一倍左右，成功優化影像處理的速度。

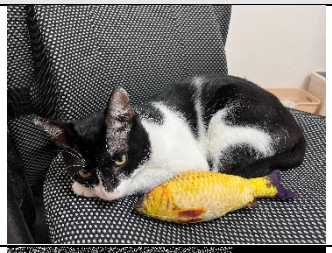
✧ Edge_Detection_Binarization_Image_Cat_Large

❖ 真實圖片（4032*3024 pixels）經過 CPU、GPU 1、GPU 2 影像處理後的結果：

1. Normal（Sobel 邊緣偵測）

Sobel Operator	CPU	GPU 1	GPU2
真實圖片			
3x3			
5x5			
7x7			

2. Binarization（邊緣偵測+Binarization）

Sobel Operator	CPU	GPU 1	GPU2
真實圖片			
3x3			
5x5			
7x7			

由以上圖片可以看到，CPU、GPU1、GPU2 的模擬圖片都相同，成功進行影像處理加速的同時也確保資料不會丟失、誤傳。

❖ 「Edge_Detection_Binarization_Image_Cat_Large」運行速度總表：

Edge_Detection_Binarization_Image_Cat_Large									
Normal					Binarization				
Pixel	Sobel Operator	GPU1 (ms)	GPU2 (ms)	CPU (ms)	Pixel	Sobel Operator	GPU1 (ms)	GPU2 (ms)	CPU (ms)
4032*3024	3×3	461792.44	153757.70	194.09	4032*3024	3×3	464428.09	151906.84	247.10
	5×5	988725.25	673643.75	212.56		5×5	994007.75	668914.50	87.68
	7×7	1763198.25	928914.56	230.54		7×7	1784863.38	920680.88	85.87
AVG		1071238.65	585438.67	212.40	AVG		1081099.74	580500.74	140.22

左邊為資料夾「Normal」中結果，僅進行 Sobel 邊緣偵測；右邊為資料夾「Binarization」中結果，進行 Sobel 邊緣偵測後再進行二值化處理，可以看到 GPU2 的速度相較於 GPU1 都快了一倍左右，成功優化影像處理的速度。

✧ Edge_Detection_Binarization_Image_Cat_Small

❖ 真實圖片（500*500 pixels）經過 CPU、GPU 1、GPU 2 影像處理後的結果：

1. Normal（Sobel 邊緣偵測）

Sobel Operator	CPU	GPU 1	GPU2
真實圖片			
3x3			
5x5			
7x7			

由以上圖片可以看到，CPU、GPU1、GPU2 的模擬圖片都相同，成功進行影像處理加速的同時也確保資料不會丟失、誤傳。

2. Binarization（邊緣偵測+Binarization）

Sobel Operator	CPU	GPU 1	GPU2
真實圖片			
3x3			
5x5			
7x7			

❖ 「Edge_Detection_Binarization_Image_Cat_Small」運行速度總表：

Edge_Detection_Binarization_Image_Cat_Small									
Normal					Binarization				
Pixel	Sobel Operator	GPU1 (ms)	GPU2 (ms)	CPU (ms)	Pixel	Sobel Operator	GPU1 (ms)	GPU2 (ms)	CPU (ms)
500*500	3×3	10626.88	3518.50	3.79	500*500	3×3	9660.12	3402.59	3.54
	5×5	20307.25	14198.68	3.40		5×5	20833.83	13918.11	18.05
	7×7	35474.55	18945.85	4.04		7×7	37345.97	19645.51	3.44
AVG		22136.23	12221.01	3.74	AVG		22613.31	12322.07	8.34

左邊為資料夾「Normal」中結果，僅進行 Sobel 邊緣偵測；右邊為資料夾「Binarization」中結果，進行 Sobel 邊緣偵測後再進行二值化處理，可以看到 GPU2 的速度相較於 GPU1 都快了一倍左右，成功優化影像處理的速度。

V. Discussion

我們在完成此專題時有遇到一些技術上的問題，在此紀錄一些相對重要的問題，並且附上我們查詢到的資料，以及我們相對應的解決方法：

✧ 執行時間會有波動

在本專題的多次模擬中，即便輸入圖片尺寸（8x8、16x16、64x64、256x256…）與 kernel 設定保持一致，每次執行所量測的時間仍然可能出現些微差異。經過上網查詢資料後，我們整理出這種波動可能主要來自以下幾個因素：

- ❖ GPU 資源共享：在共享 GPU 的開發環境中（如研究所電腦或伺服器），其他背景程序可能同時佔用 GPU 資源，導致執行順序與時間不穩定
- ❖ 記憶體傳輸延遲：資料從 Host 到 Device 的傳輸（cudaMemcpy）會受到 PCIe 匯流排繁忙程度影響
- ❖ 執行緒調度與佇列：CUDA kernel 的排程依賴底層驅動與硬體，若其他程序插入，會導致 latency 波動
- ❖ Warm-up 與 Cache 行為：首次執行 kernel 時，驅動程式可能還在設定 pipeline。而之後重複執行則受益於 Cache，導致前幾次與後幾次時間出現差異

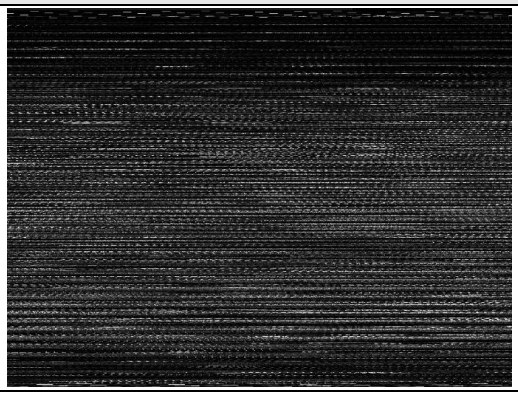
因此為了減緩這種波動，我們最後決定執行多次程式並取平均、固定執行時段避免背景干擾，如此一來可以能夠提升分析的可信度。以下附上模擬結果做為本問題的參考：

Edge_Detection_Binarization_Better							
Normal				Binarization			
Size	GPU1 (ms)	GPU2 (ms)	CPU (ms)	Size	GPU1 (ms)	GPU2 (ms)	CPU (ms)
8*8	10.44	3.06	0.06	8*8	7.22	2.95	0.05
	9.84	2.99	0.06		4.13	8.29	0.06
	4.15	8.54	0.05		4.12	8.46	0.06
	4.32	8.25	0.05		9.73	2.97	0.05
	9.65	3.07	0.06		10.58	3.15	0.06
	4.07	8.36	0.05		9.93	3.08	0.05
	9.82	2.96	0.06		21.55	3.06	0.05
	4.13	8.73	0.06		4.00	8.44	0.06
AVG	**7.05**	**5.75**	**0.05**	**AVG**	**8.91**	**5.05**	**0.05**

這是 8*8 矩陣的模擬結果，可以看到雖然我們已經加入非常好的 pipeline 切割加速方法在 GPU2 中，但是有時候還是會看見 GPU2 simulation time < GPU1 的結果發生。這是由於上述的原因造成。因此我們利用模擬多次取平均的方法得到最終模擬結果。

✧ **邊界問題 (雜訊條紋)**

我們一開始在進行真實圖片測試時曾經遇到過以下問題，圖片在 GPU 處理完後輸出的結果是橫線，經過多次測試和 debug 後才得到正確的輸出圖片：

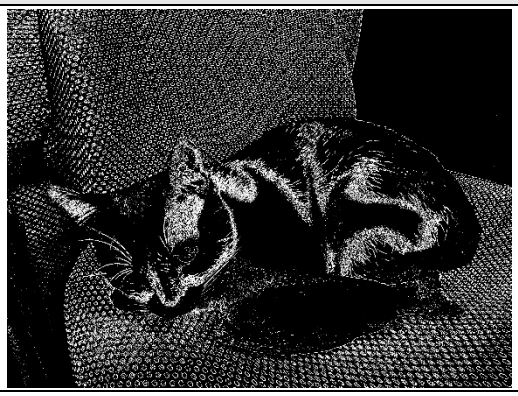
輸入的測試圖片（test.jpg）	錯誤的 GPU 輸出
	

接著是我們查詢各種資料後整理出的討論結果。在進行 Sobel 邊緣偵測時，我們對每個像素需取其周圍 3x3 區域進行卷積。然而當處理靠近圖像邊界的像素（例如最外圈的第一列與第一行），其鄰近像素不足，可能會導致以下問題：

- ❖ 讀取超出範圍的記憶體：若邊界沒有設好，會讀取錯誤位置或甚至造成程式 crash
- ❖ 雜訊條紋出現：邊界像素未正確計算，會出現明顯的白條或黑條紋

因此我們最後透過 Padding + 邊界條件判斷的方式解決此問題。具體作法如下：

- ❖ 在原圖四周加上 1 像素的 padding
  （總大小從 SIZE × SIZE 擴展為(SIZE+2) × (SIZE+2)）
- ❖ 在 kernel 中使用 if(r >= 0 && r < SIZE+2 && c >= 0 && c < SIZE+2) 判斷邊界，避免 out-of-bounds 存取
- ❖ padding 區域值設為 0，可視為對邊界進行 zero-padding，使邊緣運算具有可控性

輸入的測試圖片（test.jpg）	debug 後得到正確 GPU 輸出
	

✧ 使用 Shared Memory 與 Kernel 架構改善的整體考量

我們在進行本專題的真實圖片處理，實作 GPU 加速的 Sobel 邊緣偵測時，根據不同圖片尺寸與運算負載，設計了 CUDA2（pipeline + shared memory）。我們在設計過程中觀察到 pipeline + shared memory 會使效能表現有極大影響，也實際經歷了大量使用 Shared memory 所遭遇的挑戰，並根據不同需求進行針對性的設計與優化。

以下是針對使用 Shared Memory 以及針對不同 Kernel 改善的程式碼：

使用 Shared Memory

```
// --- Shared Memory General Version ---
__global__ void sobel_gpu_general(unsigned char* input, unsigned char* output, int width, int height, int kernelSize) {
    int offset = kernelSize / 2;
    __shared__ unsigned char smem[TILE_WIDTH + MAX_KERNEL_SIZE - 1][TILE_WIDTH + MAX_KERNEL_SIZE - 1];

    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int row_o = blockIdx.y * TILE_WIDTH + ty;
    int col_o = blockIdx.x * TILE_WIDTH + tx;
    int row_i = row_o - offset;
    int col_i = col_o - offset;

    if (row_i >= 0 && row_i < height && col_i >= 0 && col_i < width) {
        smem[ty][tx] = input[row_i * width + col_i];
    } else {
        smem[ty][tx] = 0;
    }

    __syncthreads();

    if (ty >= offset && ty < TILE_WIDTH + offset &&
        tx >= offset && tx < TILE_WIDTH + offset &&
        row_o < height && col_o < width) {

        int Gx = 0;
        int Gy = 0;

        #pragma unroll
        for (int i = 0; i < MAX_KERNEL_SIZE; ++i) {
            #pragma unroll
            for (int j = 0; j < MAX_KERNEL_SIZE; ++j) {
                int ky = i - offset;
                int kx = j - offset;

                if (i < kernelSize && j < kernelSize) {
                    int pixel = smem[ty + ky][tx + kx];
                    Gx += SOBEL_X_7x7[i * kernelSize + j] * pixel;
                    Gy += SOBEL_Y_7x7[i * kernelSize + j] * pixel;
                }
            }
        }

    int mag = (int)sqrtf((float)(Gx * Gx + Gy * Gy));
    output[row_o * width + col_o] = mag > 128 ? 255 : 0;
        //int mag = abs(Gx) + abs(Gy);
        //if (mag > 255) mag = 255;
        //output[row_o * width + col_o] = (unsigned char)mag;
    }
}
```

針對不同 Kernel

```
if (kernelSize == 3) {
    dim3 blockDim(16, 16);
    dim3 gridDim((width + 15) / 16, (height + 15) / 16);
    sobel_gpu_3x3_fast<<<gridDim, blockDim>>>(d_input, d_output, width, height);
} else {
    dim3 blockDim(TILE_WIDTH + MAX_KERNEL_SIZE - 1, TILE_WIDTH + MAX_KERNEL_SIZE - 1);
    dim3 gridDim((width + TILE_WIDTH - 1) / TILE_WIDTH, (height + TILE_WIDTH - 1) / TILE_WIDTH);
    sobel_gpu_general<<<gridDim, blockDim>>>(d_input, d_output, width, height, kernelSize);
}
```

我們採用的 Shared memory 是 CUDA 中位於 SM (Streaming Multiprocessor)的高速共享記憶體區塊，可由同一 thread block 的 thread 共同使用，具有極低的存取延遲。雖然 Shared memory 能夠顯著加速運算，但我們也在開發中遇到了多次失敗與 Debug，最終才成功實作穩定的版本。因此特地整理使用 Shared memory 的優缺點，並說明為何要依據 kernel 架構調整其使用方式。

使用 Shared memory 的好處：

- ❖ 可重用資料、減少 global memory 存取（如卷積運算中的區域重疊）
- ❖ 提升記憶體存取效率，避免 global memory 零碎問題
- ❖ 大幅提升效能，特別是在處理中大型圖（512x512、4096x4096）時，shared memory 的使用能使 GPU 計算資源發揮極大潛力，有效縮短處理時間

使用 Shared memory 的限制：

- ❖ 每個 SM 有固定 shared memory 容量，需避免超過 SM 資源（太大 kernel 無法啟動）
- ❖ 不同 thread 若需存取相同區域，透過__syncthreads()同步，否則可能出現 race condition
- ❖ block 尺寸設計需平衡效率與共享空間

根據以上分析，Shared memory 特別適用於資料重用率高的場景，但圖片尺寸過小，其管理成本與同步開銷反而會導致效能下降。因此僅在處理中大型圖片時才啟用 Shared memory，加上同步與 pipeline 機制，才能達到最佳效能。

VI. Conclusions

我們在本專題成功針對 Sobel 邊緣偵測與二值化處理進行效能優化，並驗證在不同圖像尺寸與實際應用場景中，皆能維持準確性與穩定性。我們從亂數生成矩陣開始，逐步優化演算法與記憶體配置，再延伸至多張真實圖片測試，全面評估處理效能與通用性。透過條件式 pipeline 切割策略、Shared memory 技術與合理的邊界處理，我們設計出的 CUDA2 kernel 相較於原始版本（CUDA1），在處理大尺寸圖像時可達數百倍的效能提升，而在小圖上亦保有穩定表現。

本專題不僅展示了高效 GPU 平行運算對於影像處理速度的實質貢獻，也強調了功能正確性與邊界處理的重要性。我們的最終成果不僅符合「運算量大且可平行化」的專題要求，更具備良好的擴展能力與實作價值，能應用於各式圖像前處理場景，為後續影像辨識與分類任務提供高品質的輸入資料，實現快速且可靠的系統架構。

VII. Reference

[1] 助教提供的參考講義以及影片

[2] 邊緣偵測-索伯算子（Sobel Operator）, https://reurl.cc/lz1bpl

[3] Parallel Computing (CUDA Programming), https://aandds.com/blog/parallel-computing.html